

Day03 笔记

Day03 笔记

复习和反馈

1. 字符串

1.1 字符串定义

1.2 字符串的输出和输出

1.3 下标

1.4 切片

1.5 查找方法

`find()` & `rfind()`

`index` & `rindex()`

`count()`

1.6 字符串替换方法 `replace`

1.7 字符串分隔 `split()`

1.8 字符串连接 `join()`

2. 列表

2.1 列表的定义和基本使用

2.2 遍历

2.3 向列表中添加数据

2.4 列表中的数据查询操作

2.5 列表中的删除操作

2.6 列表排序和逆置

2.7 列表嵌套

2.8 案例: 分配办公室

3. 元组

复习和反馈

素数是啥 也也太难了 还的带循环

希望老师讲课多用些例子.

`+= c+=a ==> c = c + a` 这几赋值运算符 之后前两个

1. 字符串

1.1 字符串定义

- 1 带引号的内容就是字符串.
- 2
- 3 在 python 中, 字符串可以乘上一个整数,
- 4 字符串 `*` num

```

1  # 单引号
2  name = 'isaac'
3  print(type(name), name)
4  # 双引号
5  name = "isaac"
6  print(type(name), name)
7
8  # 三引号
9  my_str = """hello world
10 hello python!
11 """
12 print(type(my_str))
13
14 my_str = '''aaa
15 bbb
16 '''
17 print(type(my_str))
18
19 # 如果字符串本身包含单引号, 使用双引号定义, 如果包含双引号可以使用单引号定义, 或者统一使用三引号引号
20 # my name is 'isaac'
21 my_name = "my name is 'isaac'"
22

```

激活 Windows

```

>>> '*' * 30
'*************************'

>>> 'hello' * 3
'hellohellohello'

```

1.2 字符串的输出和输入

```

1  # 输入 input()
2  input() 函数得到的内容就是字符串
3
4  # 输出 print() 函数 %s
5  f-string

```

1.3 下标

```

1 # 下标也称为是索引, 是一个整型数字, 可以是正数, 也可以是负数
2 # 正数下标是从0开始的, 表示第一个字符, -1 表示最后一个字符
3 my_str = 'hello'
4 # 下标的使用语法 变量[下标]
5 print(my_str[0]) # h
6 print(my_str[1]) # e
7
8 print(my_str[-1]) # o
9 print(my_str[-3]) # l
10
11 # len() 函数可以得到字符串的长度
12 print(len(my_str)) # 5
13 # 使用正数下标书写字符串中最后一个元素
14 print(my_str[len(my_str) - 1])
15

```

0	1	2	3	4	正数下标	
h	e	l	l	o		
-5	-4	-3	-2	-1	负数下标	

C:\Deve
h
e
o
l
5
o

1.4 切片

- 1 my_str[:] 得到和原来一样的字符串
- 2 my_str[::-1] 字符串的逆置

```

1 # 切片可以获取一段数据, 多个数据, 下标(索引只能获得一个数据)
2 # 切片语法: 变量[start:end:step], 会得到一个新的字符串
3 # start 开始位置的下标
4 # end 结束位置的下标, 不包含end 对应的下标
5 # step 步长, 下标之间的间隔, 默认是1
6 my_str = 'hello'
7 my_str1 = my_str[2:4:1] # ll
8 print(my_str1)
9 # step 如果是1, 即默认值, 可以不写
10 my_str2 = my_str[2:4] # ll
11 print(my_str2)
12
13 # end 位置不写, 表示是len(), 即可以取到最后一个元素
14 my_str3 = my_str[2:] # llo
15 print(my_str3)
16 # start 位置也可以省略不写, 表示是0,
17 my_str4 = my_str[:3] # hel
18 print(my_str4)

```

0	1	2	3	4	正数下标	
h	e	l	l	o		
-5	-4	-3	-2	-1	负数下标	


```

20 # start 和end 都不写,但是冒号需要写
21 my_str5 = my_str[:] # hello
22 print(my_str5)
23
24 print(my_str[-4: -1]) # ell
25 print(my_str[3:1], '1') # 没有数据,
26 # 步长可以是负数
27 print(my_str[3:1:-1], '2') # ll
28
29 print(my_str[::-1]) # 字符串的逆置, olleh
30
31 print(my_str[::2]) # 0 2 4 hlo my_str[0:5:2]
32
33

```

0	1	2	3	4	正数下标
h	e	l	l	o	
-5	-4	-3	-2	-1	负数下标

1.5 查找方法

find() & rfind()

```

1 my_str = 'hello world itcast and itcastcpp'
2
3 # find() 在字符串中查找是否存在某个字符串
4 # my_str.find(sub_str, start, end)
5 # sub_str: 要在字符串中查找的内容, 类型 str
6 # start: 开始位置, 从哪里开始查找, 默认是0
7 # end: 结束的位置, 查找到哪里结束, 默认是len()
8 # 返回值: 即方法执行的结果是什么, 如果找到 sub_str, 返回的sub_str 在 my_str 中的位置的正数下标
9 # 如果没有找到, 返回 -1
10
11 index = my_str.find('hello') # 0
12 print(index)
13 # 从下标为3的位置, 开始查找字符串 hello
14 print(my_str.find('hello', 3)) # -1
15 print(my_str.find('itcast')) # 12
16 print(my_str.find('itcast', 15)) # 23
17 # rfind() right find() 从右边(后边) 开始查找
18 print(my_str.rfind('itcast')) # 23
19
20

```

index & rindex()

```

20 # index() 在字符串中查找是否存在某个字符串
21 # my_str.index(sub_str, start, end)
22 # sub_str: 要在字符串中查找的内容, 类型 str
23 # start: 开始位置, 从哪里开始查找, 默认是0
24 # end: 结束的位置, 查找到哪里结束, 默认是len()
25 # 返回值: 即方法执行的结果是什么, 如果找到 sub_str, 返回的sub_str 在 my_str 中的位置的正数下标
26 # 如果没有找到, 会报错
27
28 print(my_str.index('hello')) # 0
29
30 # print(my_str.index('hello', 3)) # 没有找到, 代码会报错
31
32 # rindex() 从后边开始查找
33 print(my_str.index('itcast')) # 12
34 print(my_str.rindex('itcast')) # 23
35

```

```

1 my_str = 'hello world itcast and itcastcpp'
2

```

count()

```
1 my_str = 'hello world itcast and itcastcpp'
2
36 # count(sub_str, start, end) 统计出现的次数,
37 print(my_str.count('aaaa')) # 0
38 print(my_str.count('hello')) # 1
39 print(my_str.count('itcast')) # 2
40 print(my_str.count('itcast', 20)) # 1
41
```

1.6 字符串替换方法 replace

```
1 # my_str.replace(old_str, new_str, count) 字符串的替换, 将my_str中的 old_str 替换成 new_str
2 # old_str: 将要被替换的字符串
3 # new_str: 新的字符串, 替换成的字符串
4 # count: 替换的次数, 默认是全部替换
5 # 返回值: 得到一个新的字符串, 不会改变原来的字符串
6
7 my_str = 'hello world itcast and itcastcpp'
8
9 my_str1 = my_str.replace('itcast', 'itheima')
10 print('my_str:', my_str)
11 print('my_str1:', my_str1)
12
13 my_str2 = my_str.replace('itcast', 'itheima', 1) # 替换一次
14 print('my_str2:', my_str2)
15
16
17
```

1.7 字符串分隔 split()

```
1 my_str = 'hello world itcast and itcastcpp'
2
3 # my_str.split(sub_str, count) 将my_str 字符串按照sub_str 进行切割,
4 # sub_str: 按照什么内容切割字符串, 默认是空白字符, 空格, tab键
5 # count: 切割几次, 默认是全部切割
6 # 返回值: 列表 []
7
8 result = my_str.split() # 按照空白字符, 全部切割
9 print(result)
10
11 print(my_str.split('itcast'))
12 print(my_str.split('itcast', 1)) # 切割一次,
13 print(my_str.rsplit('itcast', 1))
14
Run: 08-字符串的分隔 ×
C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day03/08-字符串的分隔.py
['hello', 'world', 'itcast', 'and', 'itcastcpp']
['hello world ', ' and ', 'cpp']
['hello world ', ' and itcastcpp']
['hello world itcast and ', 'cpp']

Process finished with exit code 0
```

1.8 字符串连接 join()

```
1 # my_str.join(可迭代对象)
2 # 可迭代对象, str, 列表(需要列表中的每一个数据都是字符串类型)
3 # 将my_str 这个字符串添加到可迭代对象的两个元素之间
4 # 返回值: 一个新的字符串, 不会改变原字符串的值
5
6 my_str = '_' .join('hello') # 会把_ 加入到 hello每两个元素之间 即 h_e_l_l_o
7 print(my_str)
8 print('*' .join('hello')) # h_*_e_*_l_*_l_*_o
9
10 # 定义列表
11 my_list = ['hello', 'cpp', 'python']
12
13 print("_".join(my_list)) # hello_cpp_python
14 print("*".join(my_list)) # hello_*_cpp*_python
15 print(" ".join(my_list)) # hello cpp python
16
```

2. 列表

2.1 列表的定义和基本使用

```

1  # 列表 是python中的一种数据类型, 可以存放多个数据, 列表中的数据可以是任意类型的
2  # 列表 list , 定义使用[] 进行定义
3
4  # 定义空列表
5  my_list = []
6  print(my_list, type(my_list))
7  my_list1 = list() # 空列表
8  print(my_list1, type(my_list1))
9
10 # 定义带数据的列表, 数据元素之间使用逗号隔开
11 my_list2 = [1, 3.14, True, 'isaac']
12 print(my_list2, type(my_list2))
13 # 求列表中数据元素的个数, 即列表的长度
14 num = len(my_list2)
15 print(num)
16
17 # 列表支持下标和切片操作
18 print(my_list2[1]) # 3.14
19 print(my_list2[-1]) # isaac
20
21 print(my_list2[1:3]) # [3.14, True]

```

```

[] <class 'list'>
[] <class 'list'>
[1, 3.14, True, 'isaac'] <class 'list'>
4
3.14
isaac
[3.14, True]
[18, 3.14, True, 'isaac']
[18, 3.14, True, 'hello']
['python', 3.14, True, 'hello']

Process finished with exit code 0

```

```

23 # 下标操作和字符串中不同的是: 字符串不能使用下标修改其中的数据, 但是列表可以使用下标修改列表中的数据
24 my_list2[0] = 18
25 print(my_list2)
26
27 my_list2[-1] = 'hello'
28 print(my_list2)
29
30 my_list2[0] = 'python'
31 print(my_list2)
32
33

```

```

[] <class 'list'>
[] <class 'list'>
[1, 3.14, True, 'isaac'] <class 'list'>
4
3.14
isaac
[3.14, True]
[18, 3.14, True, 'isaac']
[18, 3.14, True, 'hello']

```

2.2 遍历

```

1 my_list = ['郭德纲', '于谦', '小岳岳', '孙越']
2
3 for i in my_list: # i 就是列表中的每一个数据
4     print(i)
5
6
7 print('*' * 30)
8
9 j = 0 # j 表示下标
10 while j < len(my_list):
11     print(my_list[j])
12     j += 1

```

```

郭德纲
于谦
小岳岳
孙越
*****
郭德纲
于谦
小岳岳
孙越

```

2.3 向列表中添加数据

```

1 # 向列表中添加数据的方法, 都是直接在原列表中进行添加的, 不会返回新的列表
2 my_list = ['郭德纲', '于谦', '小岳岳', '孙越']
3 print(my_list)
4 # 列表.append(数据) 向列表的尾部追加数据
5 my_list.append('aa')
6 print(my_list)
7 result = my_list.append(12) # 不要这样书写.
8 # print(result) # None 关键字, 表示空,
9 print(my_list)
10
11 # 列表.insert(下标, 数据) 在指定的下标位置进行添加数据
12 my_list.insert(0, 'isaac')
13 print(my_list)
14 # print(my_list.insert(5, 3.14)) 不能这样书写, None
15
16 # 列表.extend(可迭代对象) # str 列表, 会将可迭代对象中的数据逐个添加到原列表的末尾
17 my_list.extend('hel')
18 print(my_list)
19 my_list.extend([1, 'python', 3])
20 print(my_list)
21 ['郭德纲', '于谦', '小岳岳', '孙越']
22 ['郭德纲', '于谦', '小岳岳', '孙越', 'aa']
23 ['郭德纲', '于谦', '小岳岳', '孙越', 'aa', 12]

```

```

['isaac', '郭德纲', '于谦', '小岳岳', '孙越', 'aa', 12]
['isaac', '郭德纲', '于谦', '小岳岳', '孙越', 'aa', 12, 'h', 'e', 'l']
['isaac', '郭德纲', '于谦', '小岳岳', '孙越', 'aa', 12, 'h', 'e', 'l', 1, 'python', 3]

```

2.4 列表中的数据查询操作

```

1 my_list = [1, 3.14, 'isaac', False]
2 # index() 根据数据值, 查找元素所在的下标, 找到返回元素的下标, 没有找到, 程序报错
3 # 列表中没有find方法, 只有index() 方法
4 # 查找 3.14 在列表中下标
5 num = my_list.index(3.14) # 1
6 print(num)
7
8 # num1 = my_list.index(100) # 程序报错, 因为数据不存在
9
10
11 # count() 统计出现的次数
12 num3 = my_list.count(1) # 1
13 print(num3)
14
15 # in /not in 判断是否存在, 存在是True, 不存在是False, 一般和if 结合使用
16 num4 = 3.14 in my_list # True
17 print(num4)
18
19
20 num4 = 3.14 not in my_list # False
21 print(num4)
22

```

2.5 列表中的删除操作

```

1 my_list = [1, 2, 4, 5, 6, 9]
2 # 1. 根据元素的数据值删除 remove(数据值), 直接原列表中的数据
3 my_list.remove(4)
4 print(my_list) # [1, 2, 5, 6, 9]
5 # my_list.remove(4) # 程序报错, 要删除的数据不存在
6
7
8 # 2. 根据下标删除
9 # 2.1 pop(下标) 默认删除最后一个数据, 返回删除的内容
10 num = my_list.pop() # 删除最后一个数据 9
11 print(num)
12 print(my_list) # [1, 2, 5, 6]
13
14 num = my_list.pop(2) # 删除下标为2的数据即 5
15 print(num)
16 print(my_list) # [1, 2, 6]
17 # my_list.pop(10) # 删除的下标不存在, 会报错
18
19 # 2.2 del 列表[下标]
20 del my_list[1] # 删除下标为1的数据 2
21 print(my_list)
22
23 # del my_list[10] # 删除不存在的下标, 会报错
24

```

2.6 列表排序和逆置

```
1 # 想要对列表中的数据进行排序,前提是列表中的数据类型是一样的
2 my_list = [1, 5, 3, 7, 9, 6]
3
4 # 列表.sort() 直接在原列表中进行排序
5 # my_list.sort() # 默认是从小到大排序,即升序
6 # my_list.sort(reverse=True) # 通过reverse=True,从大到小排序
7 print(my_list)
8
9 # 补充: sorted(列表) 排序,不会在原列表中进行排序,会得到一个新的列表
10 my_list1 = sorted(my_list)
11 my_list2 = sorted(my_list, reverse=True)
12 print(my_list)
13 print(my_list1)
14 print(my_list2)
15
16 print("=" * 30)
17 my_list3 = ['a', 'b', 'c', 'd', 'e']
18 # 逆置,
19 my_list4 = my_list3[::-1] # 得到一个新的列表
20 print(my_list3)
21 print(my_list4)
22
23 # 在原列表直接逆置 列表.reverse()
24 my_list3.reverse()
```

激活 V

2.7 列表嵌套

```
1 school_names = [['北京大学', '清华大学'],
2                  ['南开大学', '天津大学', '天津师范大学'],
3                  ['山东大学', '中国海洋大学']]
4
5 print(school_names[1]) # ['南开大学', '天津大学', '天津师范大学']
6 print(school_names[1][1]) # 天津大学
7 print(school_names[1][1][1]) # 津
8
9 # 山东大学
10 print(school_names[2][0])
11
12 for schools in school_names:
13     # print(schools) # 列表
14     for name in schools:
15         print(name)
16
```


2.8 案例: 分配办公室

2. 应用

1. 定义学校及办公室 `[[[],[],[]]]`
2. 定义老师 `[a b]`

3. 抓阄,
for 遍历老师列表:

抓阄, 产生随机数

根据随机数, 进入对应的办公室

1 2 3
随机的分配

```
#encoding=utf-8

import random

# 定义一个列表用来保存3个办公室
offices = [[],[],[]]

# 定义一个列表用来存储8位老师的名字
names = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']

for name in names:
    index = random.randint(0,2)
    offices[index].append(name)

i = 1

for temp_names in offices:
```

```
1 import random
2
3 # 1. 定义学校列表
4 schools = [[], [], []] # 三个小列表就是三个办公室, 对应的下标 0 1 2
5 # 2. 定义老师列表
6 teachers = ["A", "B", "C", "D", "E", "F", "G", "H"]
7 # 3. 让老师抓阄,
8 # 3.1 遍历老师列表
9 for teacher in teachers:
10     # 3.2 抓阄, 产生随机数
11     num = random.randint(0, 2) # 产生的随机数, 相当于是办公室的下标
12     # 3.3 老师进入办公室, 将老师名字添加到办公室列表中
13     schools[num].append(teacher)
14
15 2 print(schools)
16
17 for office in schools:
18     print(f'该办公室老师的个数为{len(office)}, 办公室老师的名字为:')
19     for teacher in office:
20         print(teacher, end=' ')
21     print()
22
```

```
[[['A'], ['D', 'E', 'G'], ['B', 'C', 'F', 'H']]]
该办公室老师的个数为1, 办公室老师的名字为:
A
该办公室老师的个数为3, 办公室老师的名字为:
D E G
该办公室老师的个数为4, 办公室老师的名字为:
B C F H
Process finished with exit code 0
```


3. 元组

```
1 # 元组和列表非常相似，都可以存放多个数据，可以存放不同数据类型的数据
2 # 不同点：列表使用 [] 定义，元组使用 () 定义
3 # 列表中的数据可以修改，元组中的数据不能被修改
4
5 my_list = [18, 3.14, True, 'isaac'] # 列表
6
7
8 my_tuple = (18, 3.14, True, 'isaac') # 元组
9 print(my_tuple, type(my_tuple))
10
11 # 元组支持下标和切片操作
12 print(my_tuple[1]) # 3.14
13
14 # 定义空元组，没有意义
15 my_tuple1 = ()
16 print(my_tuple1, type(my_tuple1))
17 my_tuple2 = tuple()
18
19 # 定义一个数据元素的元组，数据元素后边，必须有一个逗号
20 my_tuple3 = (3) # 3 <class 'int'>
21 my_tuple4 = (3,)
22
23 print(my_tuple3, type(my_tuple3))
24 print(my_tuple4, type(my_tuple4))
```